

1. Getting started

These instructions will assume that you are comfortable with using Python in Jupyter notebooks in VS Code, and that you have installed the packages `sympy`, `numpy`, `scipy` and `plotly` using `pip` or `conda`. You will also need to import these packages and set some options. The most convenient way to do this is as follows. Distributed together with this file is a directory called `header` containing a single file `__init__.py`. You should copy this directory to be a subdirectory of the directory where you keep your notebooks for this course, then you should include the line

```
from header import *
```

as the top line of each of your notebooks.

2. Notation for variables

- 2.1 By default names like x refer to variables as used in programming languages like Python. They are expected to have a value (which might be a number or a list or something more complicated). If you try to use a variable (for example, by trying to expand $(1 + x)^4$) without assigning a value then you will get an error. If you instead want to treat x as an abstract symbol then you need to enter `x = sp.symbols('x')`. You can do this for several variables at the same time using syntax like `x,y,a,b = sp.symbols('x y a b')`.
- 2.2 If you want variables with indices like m_0, m_1, m_2 then you should enter `m = sp.IndexedBase('m')`, and then enter `m[2]` for m_2 , for example.
- 2.3 Most variables will have single-letter names, such as x or A . Note that case is significant: Python regards A as different from a , and x as different from X . It is perfectly permissible to use long names for variables, such as `eqns` or `sols` or `SpeedOfLight`. However, you may not use words that already have a special meaning in Python. In particular, you cannot call a variable `lambda`.
- 2.4 If you enter `mu = sp.symbols('mu')` then in some (but not all) contexts, `mu` will be displayed as μ . Alternatively, there are various ways to set things up so that you can enter Greek letters directly (one option is <https://wincompose.info/>). Then you can just use Greek letters like normal English letters.

3. Generalities

- 3.1 The `help()` command will give you details about any Python or SymPy object that you are interested in. Alternatively, in VS Code you can just enter a name and hover over it with your mouse and some information will appear. However, the output from `help()` is sometimes very long and overwhelming. It may be better to ask an AI a more specific question about what you are hoping to do.
- 3.2 See Section 15 for advice on some common problems.
- 3.3 If you have several commands in the same code cell then Python will only print the result of the last one by default. If you want to see other things then you should add explicit `print()` or `display()` commands.
- 3.4 If you end a command with a semicolon then Python will not print the result even if it is the last command in the cell. You can also achieve the same effect by entering `None` as the last command in the cell.
- 3.5 You can use the symbol `_` to refer to the last thing that Python calculated and displayed, and `__` to refer to the one before that, and so on. However, this mechanism cannot be used to refer to things that were calculated but not displayed. If you move around a worksheet inserting things in different places, you should remember that `_` refers to the **most recent** result, which may not be the one that you see directly above the line where you are typing.
- 3.6 It is important to understand the difference between `=` and `==` and `sp.Eq()`.
 - If we enter `x=3`, this defines x to be equal to 3, so Python will replace x by 3 whenever it sees an x .

- If we instead enter `x==3`, we are checking whether x is currently straightforwardly equal to 3. If x is an abstract symbol which has not been assigned a numerical value then `x==3` will just be `False`, even if x could be equal to 3 in some contexts.
 - Assuming that `a` and `b` have been declared to be abstract symbols, the expression `sp.Eq(2 * a, b + 1)` represents the hypothetical question “Is $2a$ equal to $b + 1$?”, which might be true or false depending on the values of a and b . Entering this question does not assign any values to `a` or `b`, and does not return a `True` or `False` answer. It simply represents the question in a symbolic form. By contrast, the expression `2 * a == b + 1` gives `False`, because the expression `2 * a` is not visibly the same as `b + 1`. In fact, even `(a + 1) * (a - 1) == a ** 2 - 1` will return `False`, because the two sides of the equation are not visibly the same, even though they become the same after expanding out the left hand side. (The right way to test whether A and B are mathematically equivalent is to enter `sp.simplify(A-B)` and see whether you get zero.) The main use for `sp.Eq()` is to introduce equations which you later intend to solve.
- 3.7 If you want to define a function (say $f(x) = x^2 + 3$) it does *not* work to enter `f(x)=x^2+3;.` See Section 10.
- 3.8 Near the top of your VS Code window you should see a menu item marked `Restart`. If you click this then Python will restart and all values that you might have assigned to variables will be removed. You might want to do this at the beginning of each new exercise, otherwise stray values left over from previous exercises can cause trouble and confusion. Note, however, that most notebooks will have a cell at the top with some `import` statements. Restarting removes all imports, so you will need to go back to your import cell and execute it again before continuing.
- 3.9 If you just want to clear a single variable (say x) you can enter `del x` instead. To clear several variables, you can enter something like `del A,x,U`.

4. Numerical approximation

- 4.1 To get an explicit numerical approximation for an expression, use the function `sp.N()` function. For example, `sp.N(sp.pi)` gives 3.14159265358979, and `sp.N(x/3)` gives $0.3333333333x$.
- 4.2 The function `sp.N()` generally gives an answer to 15 significant figures. If you want to calculate $\pi^2/6$ to 50 digits (for example), you can enter `sp.N(sp.pi**2/6, 50)`.
- 4.3 It is possible to lose accuracy for non-obvious reasons. For example, if you enter `sp.N(1/7, 100)` then Python will work from the inside out, first calculating $1/7$ to low accuracy and only then calling `sp.N()` with a request for high accuracy that can no longer be achieved. One way to fix this is to enter `sp.N(1,100)/7`, which requests high accuracy before attempting to divide by 7.
- 4.4 In some contexts, SymPy will convert fractions to approximate decimals without being asked, which is not what we want when doing symbolic mathematics. To prevent $2/3$ from being converted to 0.6666666667, we need to enter it as `sp.Rational(2, 3)` instead of `2/3`. Sometimes this is not necessary, but the rules for whether it is necessary are complicated, so it is best to err on the safe side.

5. Algebraic operations

- 5.1 Use a `*` for multiplication; for example, $2xy$ should be entered as `2*x*y`, and $(2x + 1)\sin(3x)$ should be entered as `(2*x+1)*sp.sin(3*x)`.
- 5.2 Note also that Python will treat `xy` as a two-letter name for a single variable; if you want x times y , you must enter `x*y`.
- 5.3 Brackets need to be used for algebraic grouping, just as in ordinary mathematical writing. For the product of $x + 2$ and $x - 2$ we need to enter `(x + 2) * (x - 3)`, not `x + 2 * x - 2`. Brackets used for algebraic grouping should always be round brackets, not square or curly ones.
- 5.4 Enter powers using `**`, for example `x ** 6` for x^6 . The notation `x^6` does have a meaning (the exclusive or operator applied to x and 6) but it is almost never what you want; this can unfortunately lead to some confusing error messages.
- 5.5 Bracketing is also important when raising things to powers. To raise $x + 2$ to the power $n + 5$, we need to enter `(x + 2) ** (n + 5)`, not `x + 2 ** n + 5`. The cube root of x is $x^{1/3}$. For this to work properly in SymPy, it is important to enter $1/3$ as `sp.Rational(1,3)` (which gives $1/3$ exactly) and not `1/3` (which gives 0.33333).

- 5.6 The (nonnegative) square root of x can be entered as `sp.sqrt(x)` or `x ** sp.Rational(1,2)`. If you enter `x ** (1/2)` you will only get an approximate answer.

6. Algebraic manipulation

Suppose we have a complicated expression called A , which we want to simplify or otherwise manipulate.

- 6.1 To substitute for some or all of the variables in A , use the `subs` command. For example, if A is $(a+b)^n$, you can change the b to 1 and the n to p/q by entering `A.subs({b:1,n:p/q})`, which gives $(a+1)^{p/q}$. (Note that the correct syntax here is `b:1`, not `b=1`.)
- 6.2 Often the equations to be used in `subs(...)` will come from the `solve(...)` command. For example, to find the value of $x^2 + y^2$ at the point where $x + y = 6$ and $x - y = 2$, enter `sol=sp.solve([sp.Eq(x+y,6),sp.Eq(x-y,2)])` and then `(x**2 + y**2).subs(sol)`.
- 6.3 Enter `sp.expand(A)` to expand everything out. For example, enter `sp.expand((x+y)**3)` to expand $(x+y)^3$, giving $x^3 + 3x^2y + 3xy^2 + y^3$.
- 6.4 To simplify A , try entering `sp.simplify(A)`. If this does not do what you want, you can try `sp.factor(A)`. If A involves trigonometric functions, try `sp.trigsimp(A)` or `sp.expand_trig(A)`.
- 6.5 Note that rules like $(a^4)^{1/4} = a$ are *not valid* when a is negative. The function `sp.simplify()` will not simplify expressions like this by default. However, if you declare a to be positive using `a = sp.symbols('a', positive=True)` (instead of just `a = sp.symbols('a')`) then SymPy will apply rules that are only valid for positive a . For example, `(a ** 4) ** sp.Rational(1,4)` will simplify to a . There are also more complex ways to specify that a is an integer or lies in a certain range or has other useful properties that could be used when simplifying; see the SymPy documentation about assumptions.
- 6.6 If you suspect that A is equal to some other expression (say B) and want to check this, the best way is to enter `sp.simplify(A-B)` and see if you get zero.
- 6.7 To collect together all the terms in A involving the same power of x , enter `A.collect(x)`. For example, this converts $1 + ax + bx + cx^2 + dx^2$ to $1 + (a+b)x + (c+d)x^2$.
- 6.8 If you just want the coefficient of x^2 in A then you can instead enter `A.coeff(x,2)` or `A.coeff(x**2)`. To get the constant term, you must enter `A.coeff(x,0)` — the syntax `A.coeff(x**0)` will not work.

7. Constants

- 7.1 If you enter a fraction like `2/9` then it will immediately be converted to an approximate decimal, which is not usually what you want. If you want to treat it as an exact fraction you should enter `sp.Rational(2,9)` instead.
- 7.2 The constant $\pi \simeq 3.1415926536$ should be entered as `sp.pi`. If you just enter `pi` then that will just be treated as the name of a variable with no special value or properties. Thus, if you have not introduced `pi` in any way then you will get an error.
- 7.3 The constant $e \simeq 2.718281828$ must be entered as `sp.E` (with a capital E) or `sp.exp(1)`.
- 7.4 The square root of minus one can be entered as `sp.I`.
- 7.5 In many contexts ∞ can be entered as `sp.oo`. For example, we can enter `sp.limit((1 + 2*x)/(3 + 4*x), x, sp.oo)` to find the limit of $(1+2x)/(3+4x)$ as $x \rightarrow \infty$.

8. Solving

- 8.1 To solve the equation $x^2 + 6 = 5x$ (for example), enter `sp.solve(sp.Eq(x**2+6,5*x))`. The answer is `[2,3]`, indicating that the solutions are $x = 2$ and $x = 3$. If you want to solve for an expression being equal to zero then you do not need `sp.Eq()`, you can just enter `sp.solve(x**2+6-5*x)` for example. Similarly, if you have already defined a function g , you can enter `solve(g(t),t)` to solve the equation $g(t) = 0$. It **does not work** to enter `a == b` instead of `sp.Eq(a, b)`.
- 8.2 The syntax discussed above gives the solution to $x^2 + 6 - 5x = 0$ in the form `[2,3]`. If you prefer to have the answer in the form `[[x:2],[x:3]]` (which can be more convenient if you want to substitute the result into some other expression), you should instead enter `sp.solve({x**2-5*x+6},dict=True)`.

- 8.3 The approach discussed above will give complex roots as well as real roots. For example, if we declare `x = sp.symbols('x')` and then enter `sp.solve(x**3+x,x)` we get $[0, -i, i]$. If we only want real roots we can either use `sp.real_roots(x**3+x,x)` or we can declare x using `x = sp.symbols('x',real=True)` and then just use `sp.solve()`; this will give $x = 0$ as the only real root.
- 8.4 To solve several equations in several variables, put the equations in one set of square brackets, and the variables in another set. For example, to solve $2x + 3y = 5$ and $3x + 4y = 7$ for x and y , enter
- ```
sp.solve([2*x+3*y-5,3*x+4*y-7],[x,y])
```
- 8.5 If there are no solutions, `sp.solve()` will return the empty list `[]`. For example, this happens if we try to solve the simultaneous equations  $x + y = 1$  and  $2x + 2y = 10$ .
- 8.6 If the problem involves trigonometric functions, then there will typically be infinitely many solutions, but `sp.solve()` will only list a few of them. To get all solutions, you can try using `sp.solveset()` instead. For example, we can solve the equation  $\cos(x) = 1$  by entering `sp.solveset(sp.cos(x)-1,x)`. The result will be printed as  $\{2n\pi \mid n \in \mathbb{Z}\}$ , indicating that  $\cos(x) = 1$  if and only if  $x$  is an integer multiple of  $2\pi$ . There is quite a long story about how this result is represented internally in Python, which you can read at <https://docs.sympy.org/latest/modules/sets.html>.
- 8.7 To get approximate numerical solutions, you can use `sp.nsolve()` instead of `sp.solve()`. For example, you can enter `sp.nsolve(sp.cos(x)-sp.cos(x+1),x,42)` to find a number  $x$  with  $\cos(x) = \cos(x + 1)$ , starting the search at  $x = 42$ . When using `sp.nsolve()`, you always need to specify the starting point. If you need to solve a large number of equations numerically then it will usually be best to use the `scipy` or `numpy` packages; you should only use `sp.nsolve()` for occasional numerical solutions when you are mostly working with symbolic equations.
- 8.8 Sometimes we need to solve problems like this: find  $a$  and  $b$  such that  $a(x - 1)^2 + b(x + 1)^2 = x$  for all  $x$ . For this we can enter
- ```
sp.solve_undetermined_coeffs(a*(x-1)**2+b*(x+1)**2-x,[a,b],x)
```
- Similarly, to find u such that $\sin(t + u) = \cos(t)$ for all t , we could try
- ```
sp.solve_undetermined_coeffs(sp.Eq(sp.sin(t+u),sp.cos(t)),[u],t)
```
- This does not actually work, but we can make it work by giving the solver a hint as follows:
- ```
sp.solve_undetermined_coeffs(sp.Eq(sp.expand_trig(sp.sin(t+u)),sp.cos(t)),[u],t)
```

9. Standard functions

- 9.1 Most functions can be entered using their usual names prefixed with `sp.`, for example `sp.sin(x)` or `sp.ln(y)`. If you do not want to type the prefix, you can change your initial import statement to include
- ```
from sympy import sin, cos, tan, exp, ln
```
- (for example) then you can just enter `sin(x)` instead of `sp.sin(x)`.
- 9.2 However, you must always use brackets: enter `sp.tan(x)` instead of `tan x`, and `sp.sin(2*x)` instead of `sin 2x`.
- 9.3 The absolute value of  $x$ , traditionally written as  $|x|$ , must be entered as `abs(x)`. (This is an exception to the usual pattern: `sp.abs(x)` will not work.)
- 9.4 Both `sp.ln(x)` and `sp.log(x)` refer to the natural logarithm (to base  $e$ ) of  $x$ .
- 9.5 If you want to use the logarithm to base 10 then you should enter `sp.log(x, 10)`. Similarly, enter `sp.log(x, 2)` for  $\log_2(x)$  (the logarithm to base 2) and so on.
- 9.6 The function  $e^x$  should be entered as `sp.exp(x)`.
- 9.7 The hyperbolic function  $\sinh(x) = (e^x - e^{-x})/2$  can be entered as `sp.sinh(x)`, and similarly for the functions  $\cosh(x) = (e^x + e^{-x})/2$  and  $\tanh(x) = \sinh(x)/\cosh(x)$ . If we have an expression  $A$  which involves hyperbolic functions, we can rewrite it in terms of exponentials using `A.rewrite(sp.exp)`.
- 9.8 The traditional notation  $\sin^2(x)$  refers to the square of  $\sin(x)$ , which could also be written  $(\sin(x))^2$  or  $\sin(x)^2$ . In Python this must be entered as `sp.sin(x)**2`. Similar comments apply to  $\tan^2(x)$  and so on. Note that  $\sin(x^2)$  means something different again: it is the sin of the square of  $x$ , not the square of the sin.

- 9.9 In traditional notation  $1/\sin(x)$  is sometimes written as  $\csc(x)$  or  $\operatorname{cosec}(x)$ . In Python `sp.csc(x)` will work, but `sp.cosec(x)` will not. Of course, you can also just enter `1/sp.sin(x)`. Similarly  $1/\cos(x)$  is `sp.sec(x)`.
- 9.10 In traditional notation  $\tan^{-1}(x)$  or  $\arctan(x)$  or  $\operatorname{atan}(x)$  refers to the angle  $\theta$  such that  $\tan(\theta) = x$ . This is completely different from the number  $\tan(x)^{-1} = 1/\tan(x)$ . In Python you should enter `sp.atan(x)` (not `tan^(-1)(x)` or anything like that). Similar comments apply to `asin(x)`, `atanh(x)` and so on.
- 9.11 When  $x \geq 0$ , the notation `sp.sqrt(x)` means, by definition, the positive square root of  $x$ . (There is more to say if  $x$  is negative or complex, but we pass over that here.) Thus, for example, `sp.sqrt(9)` is definitely 3 and not  $-3$ .

## 10. Defining your own functions

- 10.1 If you want to define a simple function like  $f(x) = x^2$ , enter `f = lambda x: x^2`. It **does not** work to enter `f(x)=x^2`; instead.
- 10.2 Similarly, to define the sequence  $a(n) = (-1)^n/(x+n)$ , enter `a = lambda n: (-1)**n/(x+n)`, not `a(n)=(-1)**n/(x+n)`.
- 10.3 It **does not** work to use notation like `f'(x)` for the derivative of  $f(x)$ ; see Section 12 instead.
- 10.4 You can define a function of several variables in a similar way. For example, to define  $g(a, u, v) = u^a + v^a$ , enter `g = lambda a,u,v: u**a+v**a`.
- 10.5 For longer and more complicated functions you may prefer to use the more standard Python function definition syntax, like
- ```
def f(x, y, z):
    return (x+y)**8 + (x-y)**8 + (x+z)**8 + (x-z)**8
```

11. Plotting

In this course we mostly use the `sympy` library, but that does not, by itself, support plotting. We instead use the `numpy` and `plotly` libraries to make plots. (There is also another popular library called `matplotlib` which could be used instead of `plotly`, but on balance we have found `plotly` to be preferable.) More details of everything discussed here can be found at <https://plotly.com/python/>.

- 11.1 To plot the graph $y = x^3 - x$ from $x = -2$ to $x = 2$ (for example), we would enter

```
x = np.linspace(-2, 2, 201)
y = x**3 - x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y))
fig.show()
```

The first line sets x to be an array $[x_0, x_1, x_2, \dots, x_{200}] = [-2, -1.98, -1.96, \dots, 2]$ consisting of 201 equally spaced points between -2 and 2 (inclusive). The second line sets y to be the array $[y_0, \dots, y_{200}] = [x_0^3 - x_0, \dots, x_{200}^3 - x_{200}]$ of corresponding y values. The third line `fig = go.Figure()` creates an object called `fig` representing a blank picture, to which we can add curves, shapes, text and so on. In this case we just add a single curve passing through the points with coordinates (x_i, y_i) ; this is done by the line `fig.add_trace(go.Scatter(x=x, y=y))`. Finally, the line `fig.show()` shows the picture that we have constructed. This is not always necessary, if the last line in a cell is related to plotting then Python will often decide for itself that the resulting plot should be shown, but it does not hurt to make it explicit. The graph has various interactive features which you can discover by hovering and clicking with your mouse.

- 11.2 Here is a version with some additional features. We have added options to make the curve into a thick orange dotted line. The option `name=r'$y = x^3 - x$'` specifies the name of the curve, but this is not displayed by default. The option `showlegend=True` specifies that there should be a box showing the names of all named elements in the plot. In this case we have a single named curve, and the legend is shown at the top right.

Note that the name is written in LaTeX, and we have written `name=r'...'` instead of just `name='...'`. The `r` prefix prevents special treatment of the backslash character, and is generally needed when entering strings containing LaTeX. In this case we have no backslashes so the `r` is not required, but it is safest to use it consistently.

Note also that the colour must be referred to using the US spelling (`line_color` rather than `line_colour`). We have specified the colour as `orange`, and many other normal colour names will also work. However, for more precise control one can use hex codes, such as `#40e0d0` for a certain shade of turquoise. Sites such as <https://htmlcolorcodes.com> will help you to find appropriate hex codes.

```
x = np.linspace(-2, 2, 201)
y = x**3 - x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y, name=r'$y = x^3-x$',
                        showlegend=True, line_color='orange',
                        line_dash='dot', line_width=5))
fig.show()
```

- 11.3 Various features of the overall picture can be changed using `fig.update_layout()` (before `fig.show()`). For example, the code below specifies that the plot should be 800×400 pixels, with the x axis running from $x = -3$ to $x = 3$ and the y -axis from $y = 0$ to $y = 10$.

```
x = np.linspace(-2, 2, 201)
y = x**3 - x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y, mode='lines', name=r'$y = x^3-x$', showlegend=True))
fig.update_layout(height=400, width=800, xaxis_range=[-3,3], yaxis_range=[0,10])
fig.show()
```

- 11.4 To make the vertical scale the same as the horizontal scale, add the options `yaxis_scaleanchor="x", yaxis_scaleratio=1` in `fig.update_layout()`. Alternatively, provided you have done `from header import *` as suggested at the top of this file, you can enter `equal_aspect(fig)` to make the scales equal.

- 11.5 To plot more than one graph in the same picture, we can just call `fig.add_trace()` several times. For example, we can plot $y = x^3 - x$ and $y = x^3 + x$ in the same picture as follows:

```
x = np.linspace(-2, 2, 201)
y0 = x**3 - x
y1 = x**3 + x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y0))
fig.add_trace(go.Scatter(x=x, y=y1))
fig.show()
```

If we do not specify the colours, they will be chosen automatically so that the two curves have different colours.

- 11.6 Similarly, we can plot $\cos(t)$ and $\sin(t)$ together from $t = -3\pi$ to $t = 3\pi$ as shown below. We have called `use_pi_ticks(fig, -3, 3, 2)` to put tick marks on the horizontal axis at positions from -3π to 3π with steps of $\pi/2$. If we did not do that, we would instead get tick marks at positions $-8, -6, -4, \dots, 8$, which is not very helpful when plotting functions that are periodic with period 2π .

The function `use_pi_ticks()` will only work if you have done `from header import *` as suggested at the top of this file.

Note also that we have used the function `np.sin()` (from the `numpy` library) rather than `sp.sin()`. Because `t` is an array rather than a single number, it would not work to use `sp.sin(t)`. One could instead use `[sp.sin(t0) for t0 in t]`, but that would not be efficient: the `numpy` library is highly optimised for numerical calculation, and `sympy` is not.

- 11.7 To make the x and y axes invisible (together with the associated tick marks and labels) add the options `xaxis_visible=False, yaxis_visible=False` in `fig.update_layout()`.

- 11.8 If we plot a function with discontinuities, like $y = 1/x$, then by default we will get spurious vertical lines at the discontinuity points. This can be avoided as follows:

```
x = np.linspace(-2, 2, 400)
y = 1/x
jumps = np.where(np.abs(np.diff(y)) >= 0.5)[0]
x = np.insert(x, jumps+1, np.nan)
y = np.insert(y, jumps+1, np.nan)
fig = go.Figure()
```



```
fig.add_trace(go.Scatter(x=x, y=y, mode='lines',
                        name=r'$y = \{x\}$', showlegend=True))
fig.update_layout(height=400, width=800, yaxis_range=[-8,8], showlegend=True)
```

The third line finds the values of x where y jumps by more than 0.5, indicating a discontinuity. (The value 0.5 might need to be adjusted to give the right outcome in other examples.) The next two lines insert the value `np.nan` (indicating “not a number”) at the corresponding places in the arrays x and y . The plotting system takes this as an indication that the curve should be broken without spurious vertical lines.

- 11.9 To improve the accuracy of a graph, just increase the third argument in `np.linspace`. For example, the code

```
x = np.linspace(-1, 1, 50)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=x * np.sin(np.pi / x)))
```

gives a very jagged picture, but it becomes much better if we change `np.linspace(-1,1,50)` to `np.linspace(-1,1,1000)`.

- 11.10 To plot the curve given parametrically by $x = 1/(1 + t^2)$ and $y = \sin(t)$ from $t = -5$ to $t = 5$ (for example), enter

```
t = np.linspace(-5, 5, 400)
x = 1/(1 + t ** 2)
y = np.sin(t)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y))
```

Similarly, to plot the curve $(x, y) = (\sin(2t), \cos(3t))$ for $t = 0$ to 2π , enter

```
t = np.linspace(0, 2*np.pi, 400)
fig = go.Figure()
fig.add_trace(go.Scatter(x=np.sin(2*t), y=np.cos(3*t)))
```

- 11.11 Consider a curve given implicitly, say by the equation $x + y + x^2y^2 = 1$ with $-3 \leq x \leq 3$ and $-4 \leq y \leq 4$. This can be plotted as follows:

```
x0 = np.linspace(-3,3,100)
y0 = np.linspace(-4,4,100)
x, y = np.meshgrid(x0, y0)
z = x + y + x**2 * y**2
fig = go.Figure()
fig.add_trace(go.Contour(x=x0, y=y0, z=z,
                        contours=dict(type='constraint', value=1),
                        line_color='red'))

fig.update_layout(height=600, width=600, xaxis_range=[-3,3], yaxis_range=[-4,4])
```

The first line sets `x0` to be a one-dimensional array of x -values, and similarly for the second line. The third line sets x to be a two-dimensional array of x -values, and y to be a two-dimensional array of y -values. The fourth line uses these to compute a two-dimensional array z of values of the function $x + y + x^2y^2$, then the sixth line draws the curve where $z = 1$. Similarly, you can plot the circle $x^2 + y^2 = 4$ like this:

```
x0 = np.linspace(-2,2,100)
y0 = np.linspace(-2,2,100)
x, y = np.meshgrid(x0, y0)
z = x**2 + y**2 - 1
fig = go.Figure()
fig.add_trace(go.Contour(x=x0, y=y0, z=z,
                        contours=dict(type='constraint', value=0),
                        line_color='red'))

fig.update_layout(height=600, width=600, xaxis_range=[-2,2], yaxis_range=[-2,2])
```

- 11.12 Here is an example where we create two separate pictures called `fig1` and `fig2` (which are not displayed), and then combine them to make a picture called `fig`, which is displayed.

```
x = np.linspace(-1,1,100)
y = x * (x ** 2 - 1)
z = 1 - x**4
fig1 = go.Figure()
```

```
fig1.add_trace(go.Scatter(x=x, y=y, mode='markers',
                          name=r'$y = x(x^2-1)$',
                          showlegend=True))

fig2 = go.Figure()
fig2.add_trace(go.Scatter(x=x, y=z, mode='lines',
                          name=r'$y = 1 - x^4$',
                          showlegend=True))

fig = go.Figure(data = fig1.data + fig2.data)
fig.show()
```

- 11.13 By default, `go.Scatter(x=u, y=v)` will produce a connected curve passing through the points (u_i, v_i) . Sometimes we might instead just want to show the points (u_i, v_i) without connecting points. For this, we need to add the option `mode='markers'` to `go.Scatter()`. For example, we can display the points $(11, 50), (22, 40), (33, 30), (44, 20), (55, 10)$ as follows:

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=[11,22,33,44,55], y=[50,40,30,20,10], mode='markers'))
```

Often we just want to display the y -values, and the x -values are not important, we just take them to be $0, 1, \dots, n-1$. In that context, we do not actually need to specify the x -values, we can just call something like `fig.add_trace(go.Scatter(y=[50,40,30,20,10], mode='markers'))` and the x -values will be inserted automatically.

12. Differentiation

- 12.1 To differentiate $x + \sin(x)$ with respect to x (for example), enter `diff(x+sin(x),x)`. This is equivalent to $\frac{d}{dx}(x + \sin(x))$ in traditional notation. Similarly, to differentiate $1/(1-t^2)$ with respect to t we enter `diff(1/(1-t^2),t)`.
- 12.2 To differentiate y three times with respect to x (for example), enter `diff(y,x,3)`. This is equivalent to d^3y/dx^3 in traditional notation. Similarly, to find the second derivative of $\sin(\theta)$ with respect to θ we enter `diff(sin(theta),theta,2)`.
- 12.3 If x and y are related implicitly by an equation (say $x + y + x^2y^2 = 1$) then you can find dy/dx in terms of x and y by implicit differentiation, like this:

```
sp.idiff(x+y+x^2*y^2-1,y,x)
```

- 12.4 Sometimes we just want to mention the derivative without actually working it out. We can do this by writing `sp.Derivative()` instead of `sp.diff()`. For example, entering `u = sp.Derivative(x**2, x)` gives an object u which is displayed as $\frac{d}{dx}x^2$. If you then enter `u.doit()` then SymPy will evaluate the derivative and return $2x$. Below we display the unevaluated and evaluated derivatives separately, then we illustrate how to make a LaTeX string which displays the equation relating them, namely $\frac{d}{dx}x^2 = 2x$.

```
x = sp.symbols('x')
u = sp.Derivative(x**2,x)
display(u)
display(u.doit())
display(Latex('$' + sp.latex(u) + ' = ' + sp.latex(u.doit()) + '$'))
```

Similarly, `v = sp.Derivative(t**10,t,2)` prints as $\frac{d^2}{dt^2}t^{10}$, whereas `v.doit()` or `sp.diff(t**10,t,2)` gives $90t^8$.

- 12.5 To find the Taylor series of a function $f(x)$ near a point $x = a$ to order n , enter `sp.series(f(x),x,a,n)`. Note that “order n ” means that the highest term allowed is a multiple of x^{n-1} , and all terms x^n and higher are discarded. For example, `sp.series(1/x,x,2,7)` gives the Taylor series for $1/x$ near $x = 2$ including terms up to $(x-2)^6$.
- 12.6 The command `p = sp.series(...)` command gives an answer including a term like $O(x^7)$, to indicate that there are infinitely many more terms, starting with a multiple of x^7 . We cannot do very much with this answer until we have converted it to an ordinary polynomial (without the $O(x^7)$ term on the end). We can do this by entering `p = p.removeO()`.

13. Integration

- 13.1 To calculate an indefinite integral like $\int \tan(x) dx$, enter `sp.integrate(sp.tan(x),x)`.

- 13.2 Sympy's answer will never include an arbitrary constant '+c'. In some contexts (particularly, solution of differential equations) the constant makes a difference and must be included. In other contexts it is irrelevant; sympy cannot help you to decide about this.
- 13.3 To calculate a definite integral like $\int_1^5 t^3 dt$, enter `sp.integrate(t**3, (t, 1, 5))`. Similarly, to calculate $\int_0^2 \sqrt{1+x} dx$, enter `sp.integrate(sp.sqrt(1+x), x=0..2)`.
- 13.4 It is also possible to have infinite limits. For example, $\int_{-\infty}^{\infty} (1+x^4)^{-1} dx$ can be entered as `sp.integrate(1/(1+x**4), (x, -sp.oo, sp.oo))`
- 13.5 As with differentiation, it is sometimes useful to mention an integral without evaluating it. You can do this by writing `sp.Integral()` instead of `sp.integrate()`. For example, if you enter `u = sp.Integral(x^2, x)` then you will get an object that is displayed as $\int x^2 dx$; if you then enter `u.doit()`, you will get $x^3/3$.

14. Sequences and sums

- 14.1 We might want to talk about the sequence $[1/2, 1/4, 1/6, 1/8, 1/10]$, which is the sequence of terms $1/(2k)$ for k from 1 to 5 inclusive. There are two ways to do this. The first is to use an ordinary Python list comprehension, as shown below. For this we need to use the standard Python convention for ranges: the endpoint of the range is never included, so the list $1, \dots, 5$ is `range(1, 6)` not `range(1, 5)`.

Although the cell below mentions a variable `k`, this is actually a new temporary variable that only has meaning within the list comprehension. It does not matter whether we already had a variable called `k`, that would not affect anything.

```
[sp.Rational(1, 2*k) for k in range(1,6)]
```

The second approach is to use `sp.sequence()`. This is different in various ways:

- Here the variable `k` needs to be a SymPy symbol with no assigned value.
- The range of the variable (which includes `k = 5` but not `k = 6`) is now written as `(k, 1, 5)`, matching traditional mathematical notation but not the usual Python convention for ranges.
- In this example the upper limit for `k` is the explicit integer 5, but we could also do something like `sp.sequence(1/(2 * k), (k, 1, n))` where `n` is a SymPy symbol with no assigned value. This would not be possible with an ordinary list comprehension.

We can convert a SymPy sequence to an ordinary list using the `list()` function.

```
k = sp.symbols('k')
u = sp.sequence(1/(2 * k), (k, 1, 5))
display(u)
display(list(u))
```

- 14.2 For an ordinary Python list, we can use `sum()` to find the sum of the terms.

```
u = list(10 ** k for k in range(5))
display(u)
display(sum(u))
display(sum(10 ** k for k in range(5)))
```

- 14.3 For a SymPy sequence in which the upper and/or lower limits are symbols, we need to use `sp.Sum()` instead of `sum()`. For example, we can get the geometric progression formula for $\sum_{k=0}^n 3^{-k}$ as follows:

```
k, n = sp.symbols('k n', integer=True)
sp.Sum(3**(-k), (k, 0, n)).doit()
```

Note that the naming and behaviour is a bit inconsistent with the naming and behaviour for integrals. The function `sp.Sum()` always returns an unevaluated sum which is displayed as something like $\sum_{k=0}^n 3^{-k}$; it is always necessary to use `.doit()` to find the actual value of the sum.

- 14.4 We can also handle some infinite sums. For example, it is a well-known result that $\sum_{k=1}^{\infty} k^{-2} = \pi^2/6$, and we can do this in sympy by entering `sp.Sum(k**(-2), (k, 1, sp.oo)).doit()`.

15. Some common problems

- 15.1 All abstract symbols need to be introduced using something like `x = sp.symbols('x')`. If you try to use `x` without having declared it, you will get an error like `name 'x' is not defined`. It can also happen that you give `x` a value, perhaps by entering `x = 42`, and later you forget

that you did that, and try to treat x as an abstract symbol. This might give you any of a range of different error messages, or it might just give you an answer that is incorrect or meaningless but with no error message. You can check the status of x by entering `type(x)`. If x has been declared as an abstract symbol, then `type(x)` should give `sympy.core.symbol.Symbol`. If x has been set equal to 42, then `type(x)` should give `int`. If x has not been introduced in any way, then `type(x)` will give an error. If x has been given a value that is no longer relevant, you can enter `del x` to remove it.

15.2 Remember `*`'s for multiplication.

- Enter $2(x + y)$ as `2*(x+y)`. If you instead enter `2(x+y)` then you may get a cryptic error message saying `'int' object is not callable`.
- Enter xy as `x * y`. If you instead enter `xy` then you will usually get an error message `name 'xy' is not defined`.
- For the product of a and $x + y$, enter `a*(x+y)`. To evaluate a function $f(t)$ at the point $t = x + y$, enter `f(x+y)`. If a is an object for which `a*(x+y)` makes sense, then $a(x + y)$ will not make sense and will give an error. If f is an object for which `f(x+y)` makes sense, then `f * (x + y)` will not make sense and will give an error.

15.3 The fraction $\frac{a+b}{c+d}$ must be entered as `(a+b)/(c+d)` (not `a+b/c+d`), and $\frac{a}{(u+v)(x+y)}$ must be entered as `a/((u+v)*(x+y))` (not `a/(u+v)*(x+y)`).

15.4 $a^{1/4}$ must be entered as `a ** sp.Rational(1,4)`, not `a ** (1/4)` or `a ** 1/4`.

15.5 Always use round brackets (not square or curly ones) for algebraic grouping. Square brackets are used for lists or for subscripts (e.g. x_5 is `x[5]`). Curly brackets are used for sets or Python dictionaries. Using the wrong kind of brackets will usually generate an error.

15.6 Remember to remove definitions when you have finished with them, by using `del` or clicking the **Restart** button (but remember that you need to re-execute your `import` cells after restarting). If you enter an expression like $(x + y)^5$ and SymPy converts it to something completely unrelated like $(\sin(\theta) + e^{-u})^5$, the most likely reason is that you previously defined `x=sp.sin(theta)`; and `y=sp.exp(-u)`; and you forgot to remove these definitions. You can enter `x,y = sp.symbols('x y')` to reset x and y to be symbols with no assigned value.

15.7 Remember the constant $e = 2.71828\dots$ must be entered as `sp.E` or `sp.exp(1)`, and e^x must be entered as `exp(x)`. If you enter `e ** x` instead of `sp.exp(x)`, you will most often get an error message `name 'e' is not defined`. However, if you happen to have defined e , you might get some other behaviour.

15.8 Remember that the constant $\pi = 3.14159\dots$ must be entered as `sp.pi`, not `pi`. If you try to use `pi` instead of `sp.pi` then you will usually get an error `name 'pi' is not defined`. If you want to be able to use `pi` with no prefix, you can enter `from sympy import pi` at the top of your notebook.